Guess Market — Exercise 4 (Bonus)

Web client for the existing Guess Market server — same screens and behaviour as the JavaFX desktop client

> **Open the client at http://localhost:3000**
> Start it by double-clicking `web-client\run.bat` while Tomcat with `guess-market.war` is running (exactly as in Exercise 3). run.bat opens this address in the default browser by itself.

1. Submitters and source code

Name	ID	Email
Almog Hasson	315080101	almogha2@mta.ac.il
Noa Allouche	216369140	noaallouche10@gmail.com

GitHub repository: https://github.com/AlmogHasson/GuessMarket

2. What is in the submission

File	Purpose
`web-client\run.bat`	Starts the web client and opens it in the browser. The only thing to run.
`web-client\server.js`	A small local web server (Node.js, no libraries): serves the pages and forwards the API calls to the Guess Market server.
`web-client\package.json`	Project description. There are no dependencies, so nothing is installed.
`web-client\public\`	The client itself: `index.html`, `css\app.css`, `js\*.js` and the font of the Japanese theme.
`README.pdf`	This document.

3. How to run

Environment: Windows 10 with Node.js installed (any current version, 16 or newer), and the Exercise 3 server running. No npm install, no internet connection and no other tool is needed — the client uses only Node's built-in modules and plain HTML, CSS and JavaScript.

1. Start the server exactly as described in the Exercise 3 README: `guess-market.war` in Tomcat's `webapps` folder, Tomcat started on port 8080. Nothing about the server changes for this client.
2. Double-click `web-client\run.bat`. A console window opens and the browser opens `http://localhost:3000`. Keep the console window open while using the client; closing it stops the client.
3. Log in with a user name. Events are uploaded from the JavaFX client — as the exercise defines, the web client has no Load File. Everything the desktop clients upload appears here automatically.
4. **For a second user, use a different browser or a private (incognito) window.** All tabs of one browser share one session cookie, as on any website, so one browser is one logged-in user (see 4.5).

If port 3000 is already taken, open a command window in `web-client` and run `set PORT=3001` and then `node server.js`, and browse to `http://localhost:3001`.

Why a small local server?

A page that comes from one address may not freely call another: if the browser loaded the client from a file or from port 3000 and called `localhost:8080` directly, it would block the answers (CORS) and would not keep the server's session cookie. `server.js` therefore serves the client and forwards every request under `/guess-market/` unchanged to `http://localhost:8080/guess-market`. To the browser everything comes from one address; to Tomcat the requests look exactly like the desktop client's. The server is not modified and is started as before.

4. What the web client does

The web client copies the desktop client's screens one to one — the same layout, tables, columns, buttons, messages and rules — and talks to the same servlets with the same requests.

Screen	Same as in the desktop client
Login	Name field (English letters, digits, space, _ and -), Log in, theme picker, server address. A name that is already logged in is rejected with “This user is already logged in.”
Top bar	“Acting as: NAME”, Log out, the Animations checkbox and the Light / Dark / Japanese theme picker.
Events tab	Left: every event on the server with the four filters (method, status, commission, commission type). Right: the selected event — the LMSR view (details, option values, bets, participations) or the Order Book view (both books, Bid/Ask orders, LAST / BID / ASK / MID / SPREAD, participations). Activate Event (green) and Close Event (red, with a dialog for the winning option) appear only for the event's market maker; others see “Not Started” / “Event is open”.
Users tab	Left: all other users with balance and whether they are a market maker; the logged-in user's account rows, balance and Deposit. Right: the user's events (role, status, shares) and, for the selected one, the trades, commission, winner and profit / loss.
Dialogs	Error and information messages, the winning-option choice and the deposit dialog behave like the JavaFX dialogs (for example Deposit stays disabled until the amount is valid).

4.1 Input rules

- Share fields accept only positive whole numbers; price fields accept up to two decimals. Invalid characters never appear, as with the desktop's TextFormatters. Enter in a field presses its button.
- Buttons are enabled only when the action is possible: trading only in an open event, Place order only when side, quantity and price are filled, Activate / Close only for the market maker.
- Every rule the server enforces (balance, price below d, only the MM closes…) is shown as the server's own message.

4.2 Automatic updates

Exactly like the desktop client: every 500 ms the page asks `/version`; when the number changed, it reloads the visible tab and the logged-in user's balance. A trade or an upload in any client — desktop or web — appears in every other client within about half a second.

4.3 Themes, animations and chat

The three themes of the desktop client (Exercise 2 bonus) are ported colour for colour, including the Japanese font; the choice is remembered in the browser. The Animations checkbox (off at start) turns on the same effects: theme fade, sliding tab indicator, sliding row selection, row fade-in and pressed buttons. The chat window of the Exercise 3 bonus works too, against the same server chat. The exercise does not require these; they are there so that both clients look and feel the same.

4.4 Resizing

The screens stretch with the window like the desktop SplitPanes, and the split dividers can be dragged. In a narrow window (under 1000 px) the two halves of each tab are shown one under the other, and on very narrow screens tables scroll sideways, so nothing is cut off.

4.5 Sessions in a browser

- Closing the tab logs the user out and frees the name, like closing the desktop window.
- Reloading the page (F5) keeps the user logged in.
- All tabs of one browser share one session. A second tab of the same browser can open the same user; logging in there as a different user is refused with an explanation (otherwise it would silently take over the first tab's session). Closing one of two tabs of the same user does not log the other out.
- If the session ends (for example Tomcat restarted), the page returns to the login screen with a message.

5. Code structure

Plain HTML, CSS and JavaScript modules — no framework and no build step. We chose this so the grader needs nothing but Node.js: no npm install, no download, nothing that can fail on a clean machine. The JavaScript files mirror the desktop controllers, so each part is easy to compare with its JavaFX counterpart.

File	Role (desktop counterpart)
`server.js`	Local web server: static files + forwarding /guess-market/* to Tomcat on port 8080.
`index.html`	All screens: login, top bar, Events and Users tabs, chat (login.fxml, main.fxml, top.fxml, eventLeft/Right.fxml, usersLeft/Right.fxml, chat.fxml).
`css/app.css`	The three themes as colour tokens plus all component styles (dark.css, light.css, japanese.css).
`js/api.js`	Every server call, the same endpoints and JSON bodies; turns error answers (plain text or Tomcat's HTML error page) into one readable message (HttpClientUtil).
`js/ui.js`	Table component with the row fade-in and sliding selection, dialogs, input filters, split panes, number formatting (TableView, DialogHelper, TextFormatter, RowAnimations).
`js/main.js`	Start-up, login, top bar, tabs, theme, version polling, session handling (Main, LoginController, MainController, TopController).
`js/events.js`	The Events tab: filters, list, LMSR / Order Book views, trading, Activate / Close (EventLeftController, EventRightController).
`js/users.js`	The Users tab: users, account rows, Deposit, the user's events and details (UsersLeftController, UsersRightController).
`js/chat.js`	The floating chat window (ChatController).

6. Working with AI tools

Which tool did we choose, and why? Did we try others?

We built the client with **Claude** (Anthropic). We also tried **GitHub Copilot** and **ChatGPT**. We chose Claude because it worked on the whole project at once: it read our JavaFX client — the FXML layouts, the controllers and the three CSS themes — and the servlets and DTOs, and produced a web client that matches them screen by screen and request by request. It also tested its own work in a real browser before handing it over.

Where did the AI not deliver, and where did we have to step in?

- **Requirements and decisions were ours.** We gave it the exercise document and the rule that the server (the WAR) must stay exactly as submitted in Exercise 3, and we decided the web client must have the same UX as the desktop client. The design of the local forwarding server followed from that rule.
- **Browser sessions.** The first version did not handle the browser's shared session cookie: a user whose tab was closed without Log out stayed “already logged in”, and two tabs of one browser

could act as different users. These were found while testing; we discussed the behaviour we wanted (close = log out, reload = stay logged in, one user per browser) and it was then implemented.

- **Testing on the real server.** The AI's environment could not run Tomcat, so it tested against a stand-in server built from our real engine and DTOs with the same endpoints and messages. The final check with the real Tomcat and guess-market.war on Windows was done by us.

Was there a bug or requirement the AI could not solve?

No requirement was left unsolved. The one limitation is built into browsers, not into the code: tabs of the same browser share a session cookie, so two different users need two browsers or a private window. This is documented above instead of being worked around on the server, which must not change.

How much was done by the AI alone?

Almost all of the code — we estimate about 95% — was written by the AI from our prompts, without us editing the code. Our part was the requirements, the decisions above, reviewing the screens against the desktop client, and testing. Understanding what the code does was still needed to judge the result and to describe the behaviour we wanted.

How long did it take, and how long would it have taken without AI?

Building and testing the client with the AI took about two hours. Without AI we estimate it would have taken us more than a month: we would first have had to learn HTML, CSS, JavaScript, the browser's request and cookie rules, and then rebuild every screen of the desktop client by hand.

Did we know front-end development before?

A little. Basic familiarity with HTML, CSS and JavaScript helped us read the result, check that the screens really matched the desktop client and describe precisely what to change. It would not have been enough to write this client ourselves in the available time.

Compared with the other exercises — was this way of working fun?

Yes, more fun than the other exercises. The work moved much faster, and our time went into decisions — how the client should behave, what must stay the same as the desktop — instead of typing and debugging layout code.